

---

# Relatório Técnico: Front-end Administrativo Conecta à Vida

## 1. Sobre o Projeto

O painel administrativo do **Conecta à Vida** é uma aplicação web construída em formato Single Page Application (SPA) para atuar como a interface gráfica do sistema. Desenvolvido em conjunto com a equipe (Gustavo, Gabriel, Renan e Maycon), o foco deste módulo é fornecer uma ferramenta gerencial para profissionais de saúde e administradores visualizarem métricas, controlarem os estoques de vacinas e gerenciarem as campanhas de doação de sangue de forma visual e intuitiva.

## 2. Funcionalidades

A interface administrativa possui diversas rotas e serviços mapeados para cobrir as regras de negócio:

- **Dashboard Gerencial:** Apresentação visual de estatísticas do sistema (total de pacientes, vacinas aplicadas, alertas ativos e agendamentos) com suporte a gráficos mensais interativos.
- **Gestão de Pacientes (CRUD):** Telas para cadastro, listagem, atualização e exclusão de pacientes, incluindo suporte nativo para importação em lote e exportação de dados via arquivos CSV e relatórios em PDF.
- **Carteira de Vacinação:** Visualização e registro detalhado das doses aplicadas por paciente, incluindo controle de lote, data de aplicação e qual foi o profissional de saúde responsável.
- **Controle de Campanhas e Alertas:** Gerenciamento de campanhas de saúde (com status ativo, agendado ou encerrado) e um sistema de alertas classificados por urgência para notificar os administradores.
- **Configuração de Unidade de Saúde:** Formulários para definir os horários de funcionamento, endereços e contatos do posto de atendimento.

## 3. O que foi Melhorado

A arquitetura do front-end aplica diversas práticas modernas que otimizam tanto a experiência do usuário final quanto a do desenvolvedor:

- **Ambiente de Desenvolvimento Rápido:** A adoção do Vite como ferramenta de build (substituindo o antigo Create React App) proporciona um tempo de carregamento inicial muito menor e atualizações instantâneas na tela durante a programação (HMR).
- **Segurança e Tipagem de Dados:** O uso integral do TypeScript associado à biblioteca

Zod garante que os dados preenchidos nos formulários sigam padrões estritos (como formatação de CPF e datas) antes mesmo de serem enviados para o servidor, evitando processamento desnecessário.

- **Interface Acessível e Padronizada:** A integração do Tailwind CSS com componentes primitivos do Radix UI permite estilizar o aplicativo rapidamente enquanto mantém a acessibilidade nativa (suporte a teclado e leitores de tela) para elementos complexos como modais, menus suspensos (dropdowns) e painéis de rolagem.

## 4. Tecnologias Utilizadas

A base de código utiliza um ecossistema focado no React:

- **Core:** React 19 (com react-dom) e TypeScript.
- **Roteamento:** React Router DOM (v7).
- **Build e Bundling:** Vite.
- **Estilização:** Tailwind CSS (v4) integrado com as ferramentas do Vite.
- **Componentes de Interface (UI):** Radix UI (shadcn/ui) e ícones fornecidos pelo Lucide React.
- **Gráficos:** Recharts.
- **Formulários e Validação:** React Hook Form e Zod.

## 5. Como a aplicação "puxa" os dados (Integração com a API)

O mecanismo de comunicação entre esta interface e o banco de dados é centralizado no arquivo `src/services/api.ts`.

A aplicação utiliza a função nativa `fetch` do JavaScript para disparar requisições assíncronas via HTTP diretamente para o endereço base da API, configurado localmente como `http://localhost:8080/api`.

Quando um usuário abre a tela de pacientes, por exemplo, o React chama a função `pacienteService.listarTodos()`. Essa função faz uma requisição GET para a rota `/pacientes` no Spring Boot. O servidor processa a consulta no PostgreSQL, devolve os dados em formato JSON, e o TypeScript traduz essa resposta para a interface `Paciente[]`, permitindo que o React construa a tabela de forma dinâmica na tela do navegador.

## 6. Como Instalar e Executar

Para rodar a aplicação front-end localmente na sua máquina:

1. Certifique-se de ter o **Node.js** (versão LTS recomendada) instalado.
2. Pelo terminal, navegue até a pasta do painel administrativo: `cd conecta-a-vida-administrativo`.
3. Execute o comando `npm install` para ler o arquivo `package.json` e baixar todas as dependências do projeto para a pasta `node_modules`.
4. Execute o comando `npm run dev` para iniciar o servidor de desenvolvimento do Vite.
5. O terminal informará o link local (geralmente `http://localhost:5173`). Basta abrir este link no

navegador.

## 7. Possíveis Erros e Soluções

Durante a execução do ambiente de desenvolvimento, alguns erros típicos de sistemas distribuídos podem ocorrer:

- **Erro de "Failed to fetch" (A tela carrega, mas não exibe os dados):**
  - *Causa:* O front-end tentou "puxar" as informações pelo endereço `http://localhost:8080/api`, mas o back-end (Spring Boot) não está ligado, ou a porta 8080 está ocupada.
  - *Solução:* Abra o projeto conecta-a-vida-api no terminal ou na sua IDE Java e inicie o serviço do back-end antes de usar as funções do painel administrativo.
- **Erros de CORS (Cross-Origin Resource Sharing):**
  - *Causa:* O navegador bloqueou a requisição por segurança porque a porta do front-end (ex: 5173) está tentando consumir dados de uma porta diferente (8080), e a API não autorizou explicitamente.
  - *Solução:* Certifique-se de que a configuração `@CrossOrigin` ou um filtro CORS global esteja configurado no código Java da API para liberar os acessos provenientes da porta do Vite.
- **npm ERR! code ERESOLVE ou Falhas de Dependência ao rodar npm install:**
  - *Causa:* Conflitos temporários entre versões de bibliotecas empacotadas no `package.json`.
  - *Solução:* Se o erro persistir em novas instalações, tente usar o comando `npm install --legacy-peer-deps` para contornar checagens restritas de pares.